



COS20028 – BIG DATA ARCHITECTURE AND APPLICATION

Dr Pei-Wei Tsai (Unit Convenor)
ptsai@swin.edu.au, EN508d





Week 11



Introduction to
Hive

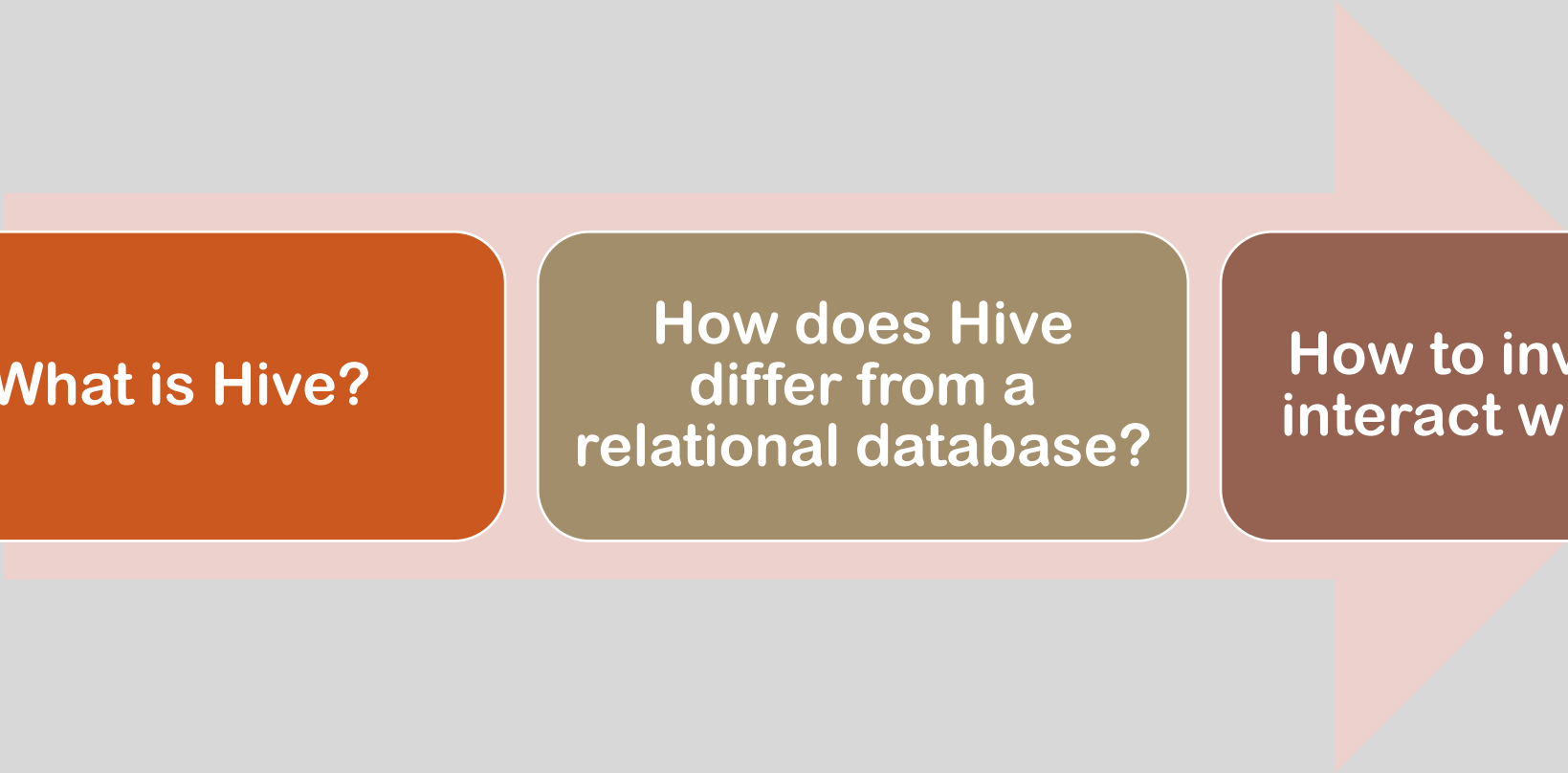
Relational Data
Analysis with
Hive





INTRODUCTION TO HIVE

Introduction to Hive



What is Hive?

**How does Hive
differ from a
relational database?**

**How to invoke and
interact with Hive?**

Overview of Apache Hive

Apache Hive is a high-level abstraction on top of MapReduce.

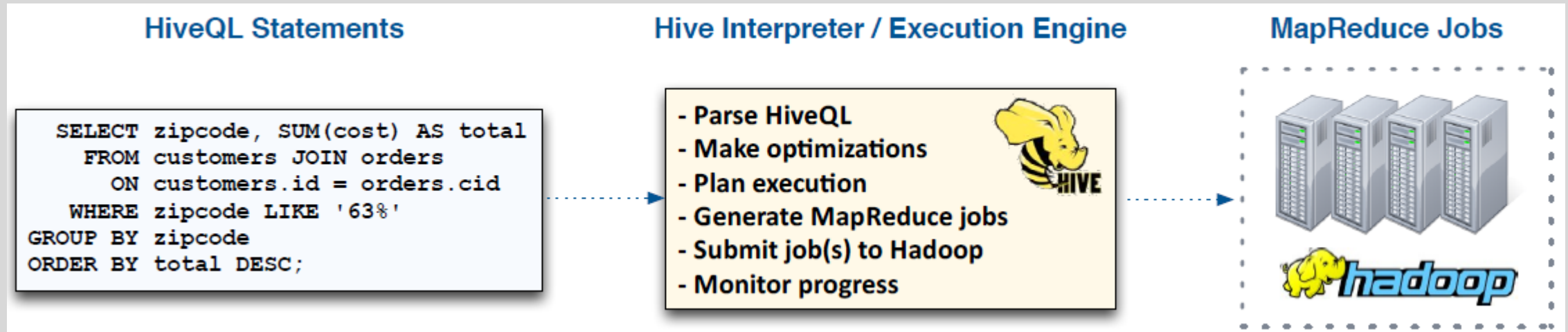
- Uses a SQL-like language called HiveQL.
- Generates MapReduce jobs that run on the Hadoop cluster.
- Originally developed by Facebook for data warehousing.
 - Now it is an open-source Apache project.

```
SELECT zipcode, SUM(cost) AS total FROM customers JOIN orders ON  
customers.cust_id = orders.cust_id WHERE zipcode LIKE '63%'  
GROUP BY zipcode ORDER BY total DESC;
```

High-Level Overview for Hive Users

Hive runs on the client machine.

- Turns HiveQL queries into MapReduce jobs.
- Submits those jobs to cluster.



Why Using Apache Hive?

More productive than writing MapReduce directly.

- Five lines of HiveQL might be equivalent to 100 lines or more of Java.

Brings large-scale data analysis to a broader audience.

- No software development experience required.
- Leverage existing knowledge of SQL.

Offers interoperability with other systems.

- Extensible through Java and external scripts.
- Many business intelligence (BI) tools support Hive.

Where to Get Hive?

CDH (Cloudera's Distribution including Apache Hadoop) is the easiest way to install Hadoop and Hive.

- A Hadoop distribution which includes core Hadoop, Pig, Hive, Sqoop, HBase, Oozie, and other ecosystem components.
- Available as RPMs, Ubuntu/Debian/SuSE packages, or a tarball.
- Simple installation.
- 100% free and open source.

Loading and Storing Data in Hive

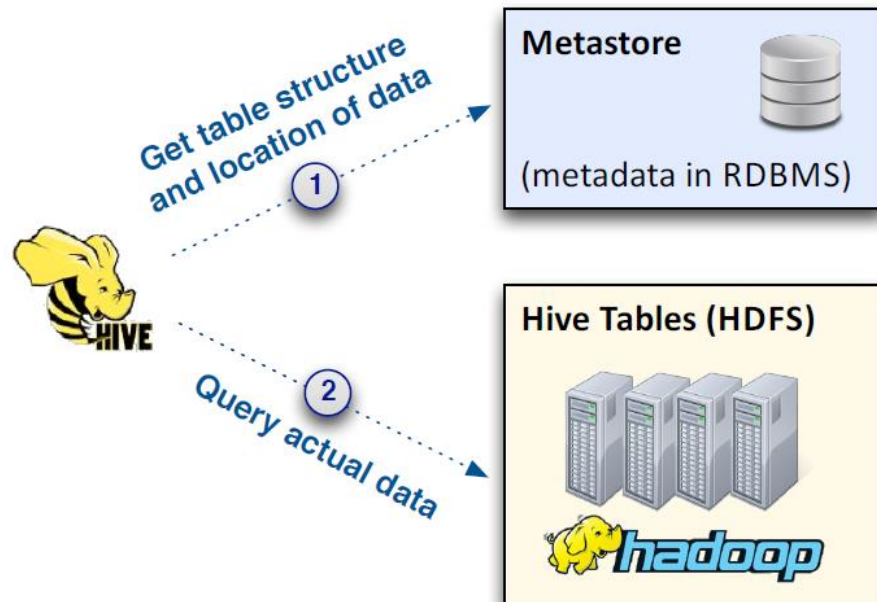
Hive's queries operate on tables, just like in an RDBMS.

- A table is simply an HDFS directory containing one or more files.
- Default path: /user/hive/warehouse/ *<table_name>*
- Hive supports many formats for data storage and retrieval.

How does Hive know the structure and location of tables?

- These are specified when tables are created.
- This metadata is stored in Hive's metastore.
 - Contained in an RDBMS such as MySQL.

Loading and Storing Data in Hive



Hive consults the metastore to determine data format and location.

- The query itself operates on data stored on a filesystem (typically HDFS).

Compare Hive with Traditional Databases

Client-server database management systems have many strengths.

- Very fast response time.
- Support for transactions.
- Allows modification of existing records.
- Can serve thousands of simultaneous clients.

Hive does not turn Hadoop cluster into an RDBMS.

- It simply produces MapReduce jobs from HiveQL queries.
- Limitations of HDFS and MapReduce still apply.

Comparing Hive to a Relational DB

	Relational Database	Hive
Query Language	SQL	HiveQL
Update Individual Records	Yes	No
Delete Individual Records	Yes	No
Transactions	Yes	No
Index Support	Extensive	Limited
Latency	Very Low	High
Data Size	Terabytes	Petabytes

Hive Use Case: Log File Analytics

Server log files are an important source of data.

Hive allows you to treat a directory of log files like a table.

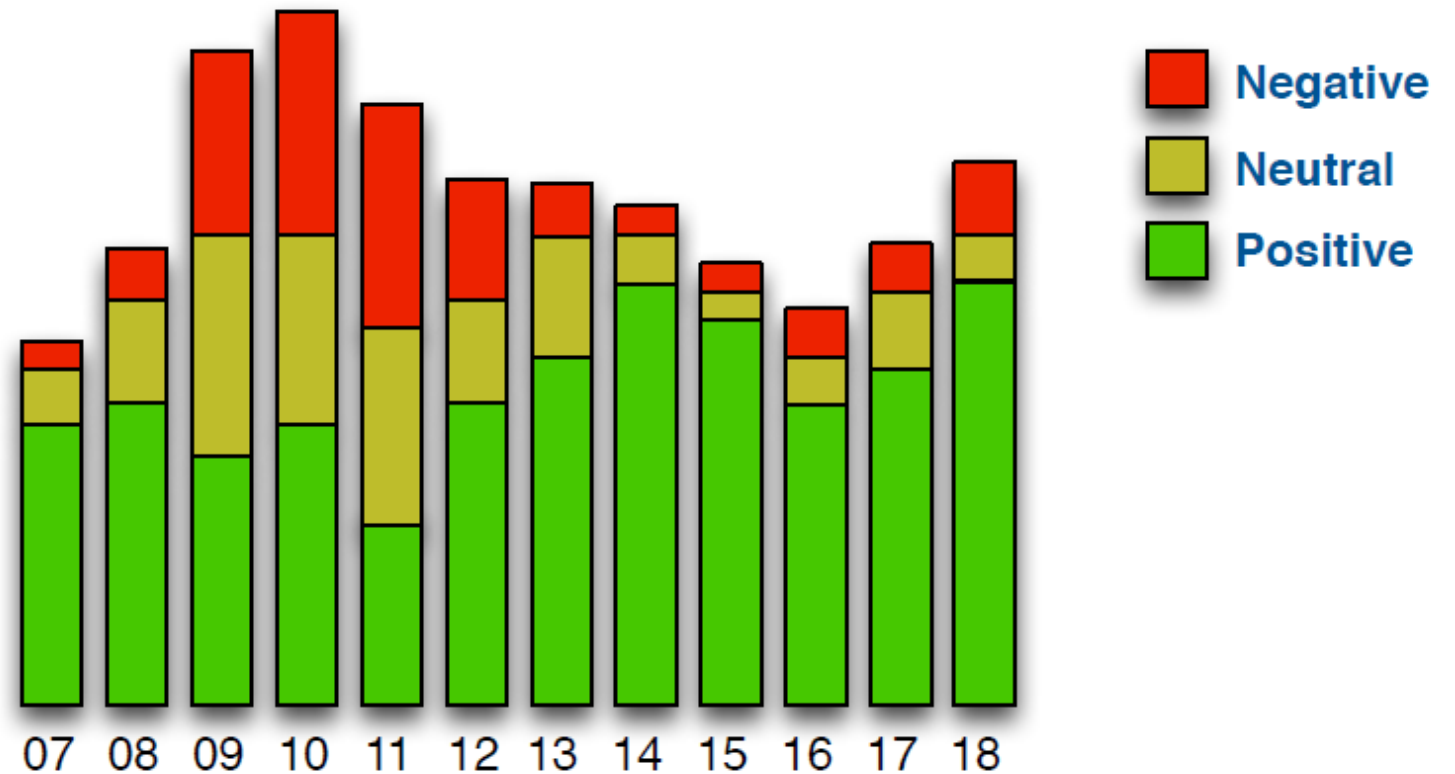
- Allows SQL-like queries against raw data.

Dualcore Inc. Public Web Site (June 1 - 8)					
Product	Unique Visitors	Page Views	Average Time on Page	Bounce Rate	Conversion Rate
Tablet	5,278	5,894	17 seconds	23%	65%
Notebook	4,139	4,375	23 seconds	47%	31%
Stereo	2,873	2,981	42 seconds	61%	12%
Monitor	1,749	1,862	26 seconds	74%	19%
Router	987	1,139	37 seconds	56%	17%
Server	314	504	53 seconds	48%	28%
Printer	86	97	34 seconds	27%	64%

Hive Use Case: Sentiment Analysis

- Many organizations use Hive to analyse social media coverage.

Mentions of Dualcore on Social Media (by Hour)





INTERACTING WITH HIVE

Using the Hive Shell

You can execute HiveQL statements in the Hive Shell.

- This interactive tool is similar to the MySQL shell.

Run the *hive* command to start the Hive shell.

- The Hive shell will display its `hive>` prompt
- Use the *quit* command to exit the Hive Shell.

```
$ hive
hive> SELECT cust_id, fname, lname FROM customers WHERE zipcode = 20525;
1000000      Quentin      Shepard
1000001      Brandon      Louis
1000002      Marilyn      Ham
hive> quit;
$
```

Accessing Hive from the Command Line

You can also execute a file containing HiveQL statements using the -f option:

```
$ hive -f myquery.hql
```

Or use HiveQL directly from the command line using the -e option:

```
$ hive -e 'SELECT * FROM users'
```

Use the -S (silent) option to suppress informational messages

- Can also be used with -e or -f options.

```
$ hive -S -f myquery.hql
```

Hive Properties

Many aspects of Hive's behaviour are configured through properties.

- Use `set -v` in Hive to see current values.

```
hive> set -v;
```

You can also use `set` to specify property values.

- The following enables column headers in query results.

```
hive> set hive.cli.print.header=true;
```

Hive runs the `.hiverc` file in your home directory at startup.

- Useful for specifying per-user defaults.

Interacting with the OS and HDFS

Use ! to execute system commands from within Hive.

- Neither pipes nor globs (wildcards) are supported.

```
hive> ! date;  
Thu Oct 23 11:34:52 EDT 2020
```

Prefix HDFS commands with dfs to use them from within Hive.

```
hive> dfs -mkdir reports/sales/2019;
```

Accessing Hive with Hue

Alternatively, you can access Hive through Hue.

- Web-based UI for many Hadoop-related services, including Hive.

To use Hue, browse to `http://hue_server:8888/`

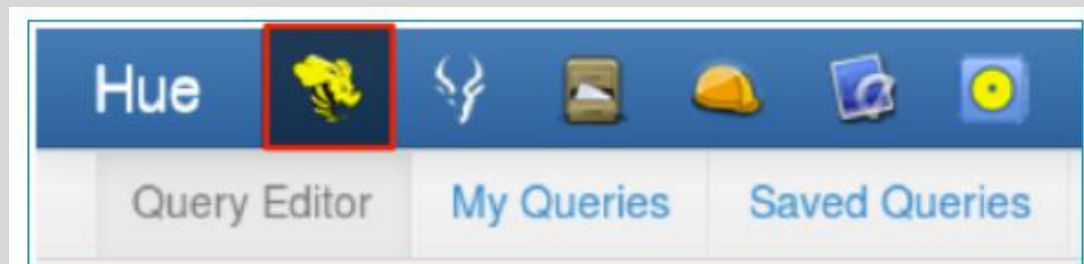
- May need to start Hue service first (`sudo service hue start`)

Hue's Hive interface is called Beeswax

- Launch by clicking its icon.

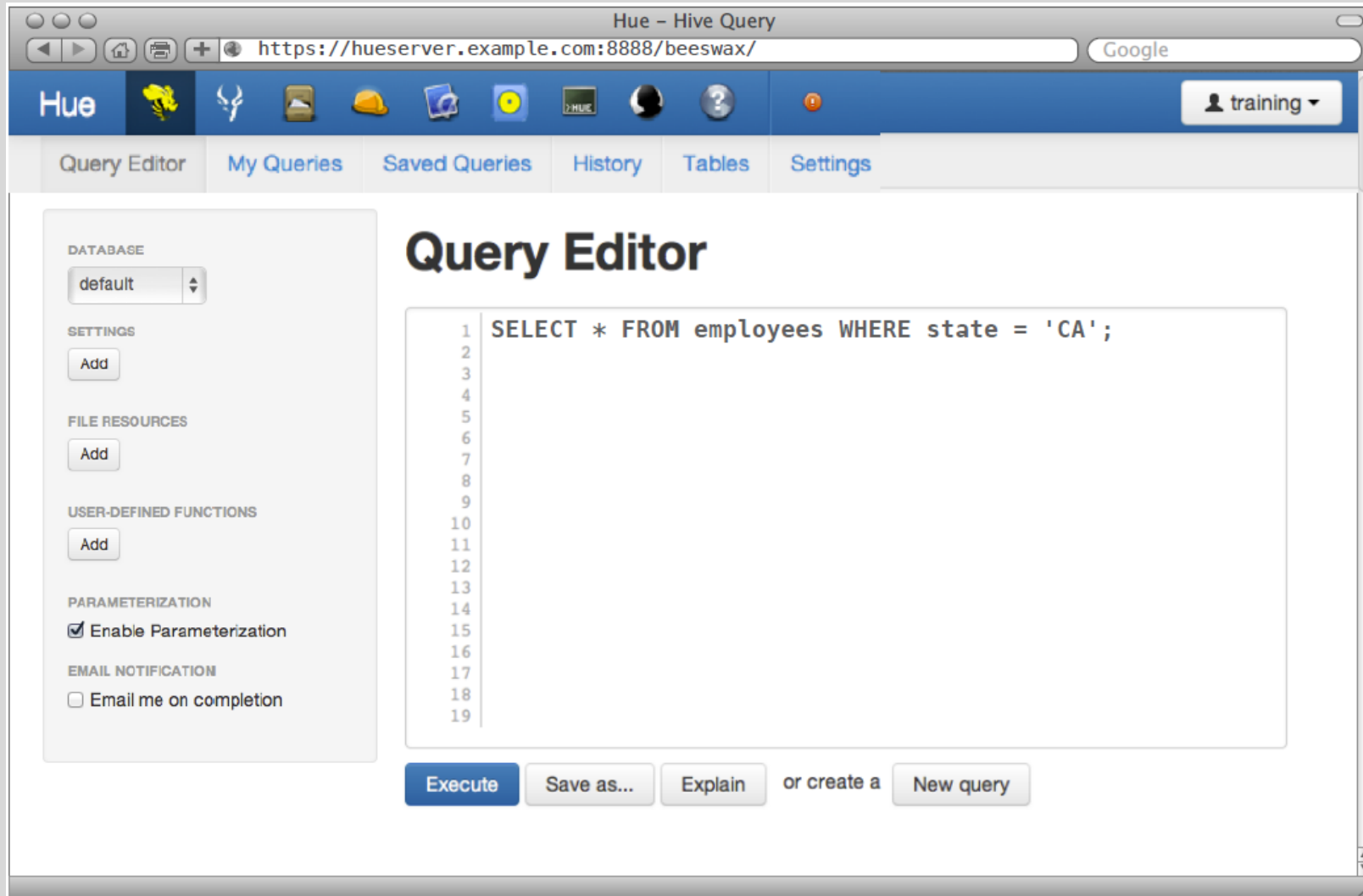
Beeswax features include:

- Creating tables.
- Running queries.
- Browsing tables.
- Saving queries for later execution.



Beeswax icon in Hue

Hue's Beeswax Query Editor



Query Results in Beeswax

Query Results: **Unsaved Query**

DOWNLOADS

<

[Download as CSV](#)

[Download as XLS](#)

[Save](#)

MR JOB (1)

[job_201305160220_0008](#)

Did you know? You can click on a row to select a column you want to jump to.

×

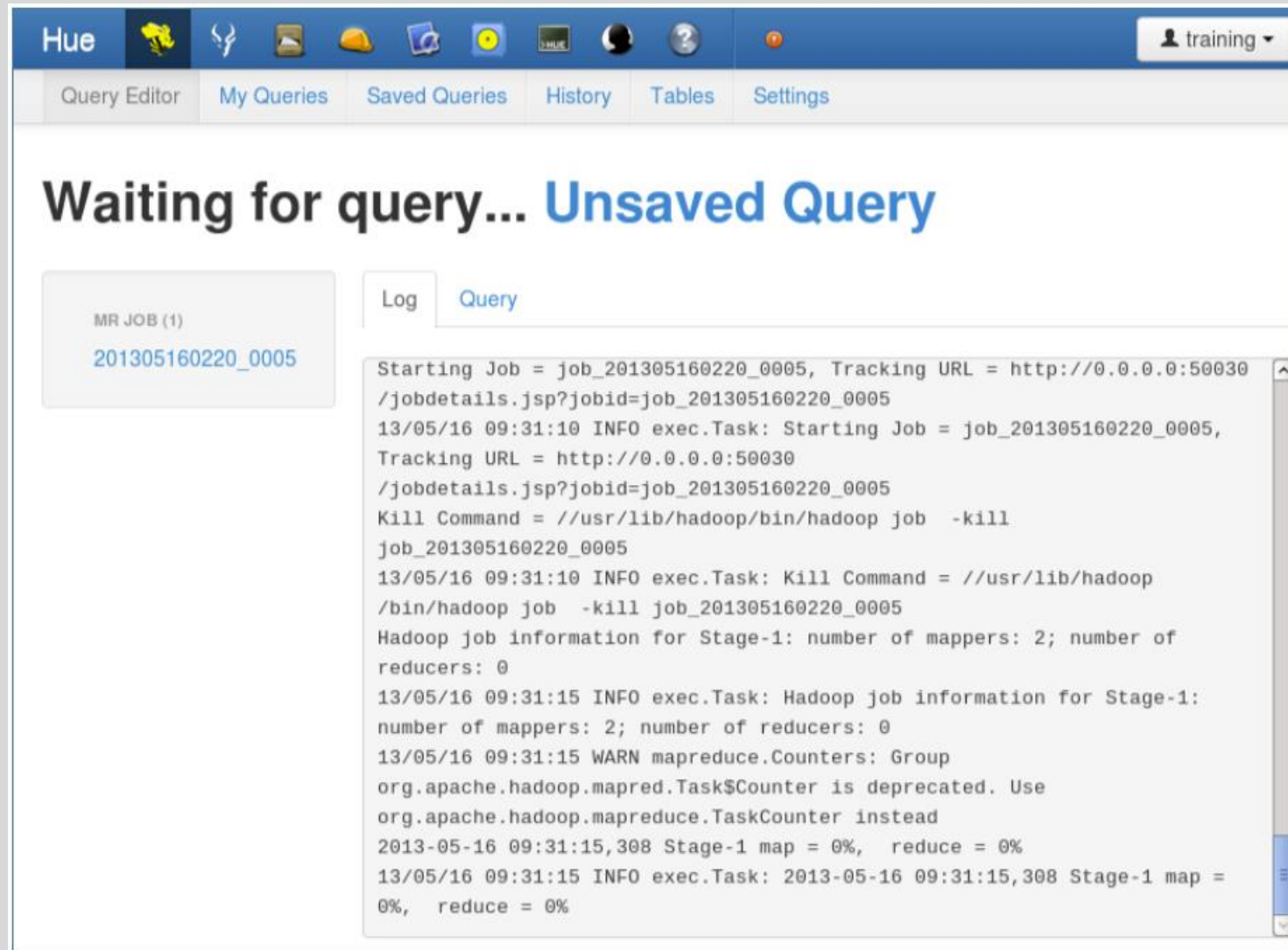
ResultsQueryLogColumns

◆ fname	◆ lname	◆ address	◆ city	◆ state	◆ zipcode
Angelica	Bullington	4593 East Richview Street	Sacramento	CA	94247
Aaron	Bullock	793 West 25th Street	Sacramento	CA	94291
Abraham	Bullock	1910 North 12th Street	Davis	CA	95616
Ashley	Burke	7382 Lewis Road	Mather	CA	95655
Alan	Burnett	885 North 15th Street	Dos Rios	CA	95429
Alphonse	Busby	992 North Clark Road	Richmond	CA	94804
Albert	Bush	478 White Road	Stockton	CA	95205
Albert	Bustamante	8489 Mason Street	San Jose	CA	95122

← Beginning of List

Next Page →

Query Execution in Beeswax



The screenshot shows the Hue web interface. At the top is a blue navigation bar with the 'Hue' logo and several icons. Below this is a secondary navigation bar with tabs: 'Query Editor', 'My Queries', 'Saved Queries', 'History', 'Tables', and 'Settings'. The main content area has a header 'Waiting for query... Unsaved Query'. On the left, there is a box labeled 'MR JOB (1)' containing the job ID '201305160220_0005'. To the right of this box are two tabs: 'Log' and 'Query'. The 'Log' tab is active, displaying a scrollable log of Hadoop job execution details. The log text includes job tracking URLs, timestamps, and progress information for Stage-1.

MR JOB (1)
201305160220_0005

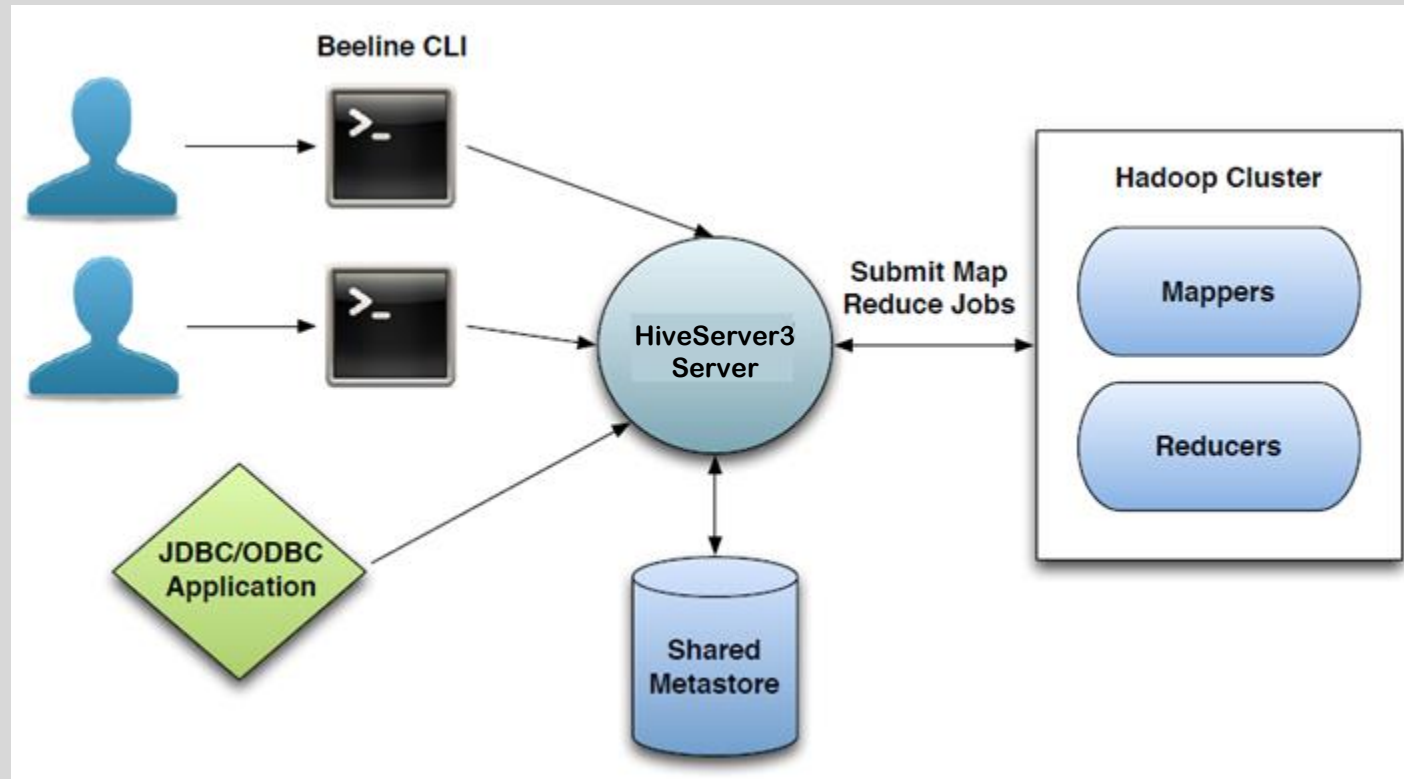
Log Query

```
Starting Job = job_201305160220_0005, Tracking URL = http://0.0.0.0:50030/jobdetails.jsp?jobid=job_201305160220_0005
13/05/16 09:31:10 INFO exec.Task: Starting Job = job_201305160220_0005, Tracking URL = http://0.0.0.0:50030/jobdetails.jsp?jobid=job_201305160220_0005
Kill Command = //usr/lib/hadoop/bin/hadoop job -kill job_201305160220_0005
13/05/16 09:31:10 INFO exec.Task: Kill Command = //usr/lib/hadoop/bin/hadoop job -kill job_201305160220_0005
Hadoop job information for Stage-1: number of mappers: 2; number of reducers: 0
13/05/16 09:31:15 INFO exec.Task: Hadoop job information for Stage-1: number of mappers: 2; number of reducers: 0
13/05/16 09:31:15 WARN mapreduce.Counters: Group org.apache.hadoop.mapred.Task$Counter is deprecated. Use org.apache.hadoop.mapreduce.TaskCounter instead
2013-05-16 09:31:15,308 Stage-1 map = 0%, reduce = 0%
13/05/16 09:31:15 INFO exec.Task: 2013-05-16 09:31:15,308 Stage-1 map = 0%, reduce = 0%
```

Interacting with HiveServer3 – Hive as a Service

HiveServer3 can be deployed to provide a centralized Hive service.

- Uses a JDBC or ODBC connection.
- Supports Kerberos authentication.



Interacting with HiveServer3 – Hive as a Server

To connect to HiveServer3, use Hue or the Beeline CLI

- You cannot use the Hive Shell.
- For secure deployments, supply your user ID and password.

Example: starting beeline and connecting to HiveServer

- Our Virtual Machine installed HiveServer2, but the usage is the same.

```
[training@localhost analyst]$ beeline
Beeline version 0.10.0-cdh4.2.1 by Apache Hive

beeline> !connect jdbc:hive2://localhost:10000
training mypwd org.apache.hive.jdbc.HiveDriver

Connecting to jdbc:hive2://localhost:10000
Connected to: Hive (version 0.10.0)
Driver: Hive (version 0.10.0-cdh4.2.1)
Transaction isolation: TRANSACTION_REPEATABLE_READ

beeline>
```

Major Differences Between HiveServer 2 and HiveServer 3

The owner of the managed tables are all *hive* after upgrading

- The ownership of the Hive table file is a factor in determining table types and ACID operations after the upgrade.
- ACID stands for atomic, consistent, isolation, and durable (ACID) properties.

The tables in HiveServer3 are all transaction tables

- HiveServer2 uses ACID v1 tables but HiveServer3 uses ACID v2 tables.
- To upgrade the ACID table, simply use the command below.

```
hive> ALTER TABLE <your_table> SET TBLPROPERTIES ('transactional' = 'true')
```

Brief Summary

Hive is a high-level abstraction on top of MapReduce.

- Runs MapReduce jobs on Hadoop based on HiveQL statements.
- Originally developed for data warehousing at Facebook.

HiveQL is very similar to SQL.

- Easy to learn for those with relational database experience.
- However, Hive DOES NOT replace your RDBMS.

Hive tables are really directories of files in HDFS.

- Information about those tables is kept in Hive's metastore.

Hive and Pig are similar in many ways.

- But each also has its own distinct advantages.

Related Information

The following offer more information on topics discussed in this lecture.

Hive Web Site

- <http://hive.apache.org/>

Beeline CLI reference

- <http://sqlline.sourceforge.net/>

Programming Hive (book)

- <http://tiny.cloudera.com/dac09a>

Data Analysis with Hadoop and Hive at Orbitz

- <http://tiny.cloudera.com/dac09b>

Sentiment Analysis Using Apache Hive

- <http://tiny.cloudera.com/dac09c>



RELATIONAL DATA ANALYSIS WITH HIVE

Hive Tables

Data for Hive's tables are stored on the filesystem (typically HDFS).

- Each table maps to a single directory.

A table's directory may contain multiple files.

- Typically comma delimited text files, but Hive supports many formats.
- Subdirectories are not allowed.

Hive uses the metastore to give context to this data.

- Helps mapping raw data in HDFS to named columns of specific types.

Hive Databases

Each Hive table belongs to a specific database.

Early versions of Hive supported only a single database.

- It placed all tables in the same database (named *default*).
- This is still the default behaviour.

Hive supports multiple databases as of release 0.7.0

- Helpful for organisation and authorisation.

Exploring Hive Databases and Tables

Which databases are available?

```
hive> SHOW DATABASES;  
accounting  
default  
dualcore  
movielens
```

```
mysql> SHOW DATABASES;  
accounting  
dualcore  
movielens  
mysql
```

Switch between databases with the *USE* statement

```
$ hive  
hive> SELECT * FROM customers;  
hive> USE sales;  
hive> SELECT * FROM customers;
```

Queries table in the *default* database.

Queries table in the *sales* database.

Exploring Hive Databases and Tables

Which tables does the current database contain?

```
hive> USE dualcore;  
hive> SHOW TABLES;  
customers  
employees  
...
```

```
mysql> USE dualcore;  
mysql> SHOW TABLES;  
customers  
employees  
...
```

Which tables are contained in a different database?

```
hive> SHOW TABLES IN movielens;  
genre  
movie  
moviegenre  
movierating  
occupation  
user
```

```
mysql> SHOW TABLES IN movielens;  
genre  
movie  
moviegenre  
movierating  
occupation  
user
```

Exploring Hive Databases and Tables

The *DESCRIBE* statement displays the schema of a table.

```
hive> DESCRIBE orders;
cust_id      int
cust_name    string
active       boolean
created      timestamp
```

```
mysql> DESCRIBE orders;
cust_id      int(11)
fname        varchar(15)
active       boolean
created      datetime
```

DESCRIBE FORMATTED shows more detailed information.

```
hive> DESCRIBE FORMATTED orders;
# col_name    data_type    comment
cust_id       int          None
cust_name     string       None
active        boolean    None
created       timestamp  None
# Detailed Table Information ...
```

```
# Detailed Table Information
Database:      accounting
Owner:         training
CreateTime:    Sat Jun 07 00:02:53 EDT 2014
LastAccessTime: UNKNOWN
Protect Mode:  None
Retention:     0
Location:      hdfs://0.0.0.0:8020/user/hive/warehouse/accounting.db/ac
counts
Table Type:    MANAGED_TABLE
Table Parameters:
    transient_lastDdlTime 1402113773

# Storage Information
SerDe Library: org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
InputFormat:   org.apache.hadoop.mapred.TextInputFormat
OutputFormat:  org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputForm
at
Compressed:    No
Num Buckets:   -1
Bucket Columns: []
Sort Columns:  []
Storage Desc Params:
```

Basic HiveQL Syntax

HiveQL is Hive's query language.

- Based on a subset of SQL-92, plus Hive-specific extensions.

Some limitations compared to 'standard' SQL

- Some features are not supported.
- Others are only partially implemented.

HiveQL also has some features not offered in SQL.

HiveQL Basics

Hive keywords are not case-sensitive

- Though they are often capitalised by convention.

Statements are terminated by a semicolon

- A statement may span multiple lines.

Comments begin with -- (double hyphen)

- Only supported in Hive scripts.
- There is no multi-line comments in Hive.

```
$ cat myscript.hql
```

```
SELECT cust_id, fname, lname FROM customers WHERE zipcode='60601'; -- downtown
```

HiveQL Basics – Selecting Data

The *SELECT* statement retrieves data from Hive tables.

- Can specify an ordered list of individual columns.

```
hive> SELECT cust_id, fname, lname FROM customers;
```

- An asterisk matches all columns in the table.

```
hive> SELECT * FROM customers;
```

Limiting and Sorting Query Results

The *LIMIT* clause sets the maximum number of rows returned.

```
hive> SELECT cust_id, fname, lname FROM customers LIMIT 10;
```

Caution: no guarantee regarding *which* 10 results are returned!

- Use *ORDER BY* for top-N queries.
- The field(s) you *ORDER BY* must be selected.

```
hive> SELECT cust_id, fname, lname FROM customers  
ORDER BY cust_id DESC LIMIT 10;
```

Using a *WHERE* Clause to Restrict Results

WHERE clauses restrict rows to those matching specified criteria.

- String comparisons are case-sensitive.

```
hive> SELECT * FROM orders WHERE order_id=1287;
```

```
hive> SELECT * FROM customers WHERE state IN ('CA', 'OR', 'WA', 'AZ');
```

You can combine expressions using *AND* or *OR*.

```
hive> SELECT * FROM customers WHERE fname LIKE 'Ann%'  
      AND (city='Melbourne' OR city='Sydney');
```

Table Aliases

Table aliases can help simplify complex queries.

```
hive> SELECT o.order_date, c.fname, c.lname FROM customers c JOIN orders o  
ON c.cust_id = o.cust_id WHERE c.zipcode='94306';
```

- Note: Using *AS* to specify table aliases is *not* supported.

Combining Query Results with *UNION ALL*

Unifies output from SELECTs into a single result set

- The name, order, and types of columns in each query must match.
- Hive only supports *UNION ALL*

```
SELECT emp_id, fname, lname, salary FROM employees WHERE state='CA'  
AND salary > 75000 UNION ALL SELECT emp_id, fname, lname, salary FROM  
employees WHERE state != 'CA' AND salary > 50000;
```

Union All can also be used with subqueries.

Subqueries in Hive

Hive only supports subqueries in the *FROM* clause of the *SELECT* statement

- It does not support correlated subqueries.

```
SELECT prod_id, brand, name FROM  
  (SELECT * FROM products WHERE (price - cost) / price > 0.65 ORDER BY  
   price DESC LIMIT 10) high_profits WHERE price > 1000  
ORDER BY brand, name;
```

Hive allows arbitrary levels of subqueries

- Each subquery **must be named** (like *high_profits* above).

Hive's Data Types

Each column in Hive is associated with a data type.

Hive supports more than a dozen types.

- Most are similar to those found in relational databases.
- Hive also supports three complex types.

Use the *DESCRIBE* command to see a table's column types.

```
hive> DESCRIBE orders;  
cust_id      int  
cust_name    string  
active       boolean  
created      timestamp
```

Hive's Integer Types

- Integer types are appropriate for whole numbers.
 - Both positive and negative values are allowed.

Name	Description	Example Value
TINYINT	Range: -128 to 127	12
SMALLINT	Range: -32,768 to 32,767	438
INT	Range: -2,147,483,648 to 2,147,483,647	99454772
BIGINT	Range: ~ -9.2 quintillion to ~ 9.2 quintillion	699234520185390

Hive's Decimal Types

- **Decimal types are appropriate for floating point numbers.**
 - Both positive and negative values allowed.
 - Caution: avoid using when exact values are required!

Name	Description	Example Value
FLOAT	Decimals	3.14159
DOUBLE	Vary precise decimals	3.14159265358979323846

Other Simple Types in Hive

- **Hive can also store several other types of information**
 - Only one character type (variable length)

Name	Description	Example Value
STRING	Character sequence	Betty F. Smith
BOOLEAN	True or False	TRUE
TIMESTAMP	Instant in time	2020-10-17 23:41:58
BINARY	Raw bytes	N/A

Complex Column Types in Hive

- **Hive also has a few complex data types**
 - These are capable of holding multiple values.

Column Type	Usage in Query
ARRAY Ordered list of values, all of the same type	departments[0]
MAP Key-value pairs, each of the same type	employees['BF5314']
STRUCT Named fields, of possibly mixed types	address.street

Joins in Hive

Joining disparate data sets is a common operation in Hive

Hive supports several types of joins

- Inner joins
- Outer joins (left, right, and full)
- Cross joins (supported in Hive 0.10 and later)
- Left semi joins

Only equality conditions are allowed in joins

- Valid: `customers.cust_id = orders.cust_id`
- Invalid: `customers.cust_id <> orders.cust_id`
- Outputs records where the specified key is found in each table.

For best performance, list the largest table last in your query

Join Syntax

Hive requires the following syntax for joins

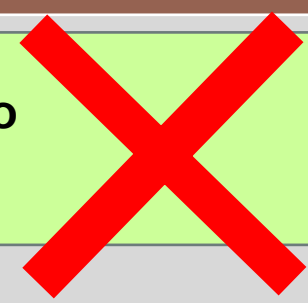
```
SELECT c.cust_id, name total FROM customers c  
JOIN orders o ON (c.cust_id = o.cust_id);
```

The above example is an inner join

- Can replace *JOIN* with another type. (e.g. *RIGHT OUTER JOIN*)

The following join syntax is NOT valid in Hive

```
SELECT c.cust_id, name total FROM customers c, orders o  
WHERE (c.cust_id = o.cust_id);
```



Inner Join Example

customers table

cust_id	name	country
a	Alice	us
b	Bob	ca
c	Carlos	mx
d	Dieter	de

orders table

order_id	cust_id	total
1	a	1539
2	c	1871
3	a	6352
4	b	1456
5	z	2137

```
SELECT c.cust_id, name, total
FROM customers c
JOIN orders o
ON (c.cust_id = o.cust_id);
```

Result of query

cust_id	name	total
a	Alice	1539
a	Alice	6352
b	Bob	1456
c	Carlos	1871

Left Outer Join Example

customers table

cust_id	name	country
a	Alice	us
b	Bob	ca
c	Carlos	mx
d	Dieter	de

orders table

order_id	cust_id	total
1	a	1539
2	c	1871
3	a	6352
4	b	1456
5	z	2137

```
SELECT c.cust_id, name, total
FROM customers c
LEFT OUTER JOIN orders o
ON (c.cust_id = o.cust_id);
```

Result of query

cust_id	name	total
a	Alice	1539
a	Alice	6352
b	Bob	1456
c	Carlos	1871
d	Dieter	NULL

Right Outer Join Example

customers table

cust_id	name	country
a	Alice	us
b	Bob	ca
c	Carlos	mx
d	Dieter	de

orders table

order_id	cust_id	total
1	a	1539
2	c	1871
3	a	6352
4	b	1456
5	z	2137

```
SELECT c.cust_id, name, total
FROM customers c
RIGHT OUTER JOIN orders o
ON (c.cust_id = o.cust_id);
```

Result of query

cust_id	name	total
a	Alice	1539
a	Alice	6352
b	Bob	1456
c	Carlos	1871
NULL	NULL	2137

Full Outer Join Example

customers table

cust_id	name	country
a	Alice	us
b	Bob	ca
c	Carlos	mx
d	Dieter	de

orders table

order_id	cust_id	total
1	a	1539
2	c	1871
3	a	6352
4	b	1456
5	z	2137

```
SELECT c.cust_id, name, total
FROM customers c
FULL OUTER JOIN orders o
ON (c.cust_id = o.cust_id);
```

Result of query

cust_id	name	total
a	Alice	1539
a	Alice	6352
b	Bob	1456
c	Carlos	1871
d	Dieter	NULL
NULL	NULL	2137

Using an Outer Join to Find Unmatched Entries

customers table

cust_id	name	country
a	Alice	us
b	Bob	ca
c	Carlos	mx
d	Dieter	de

orders table

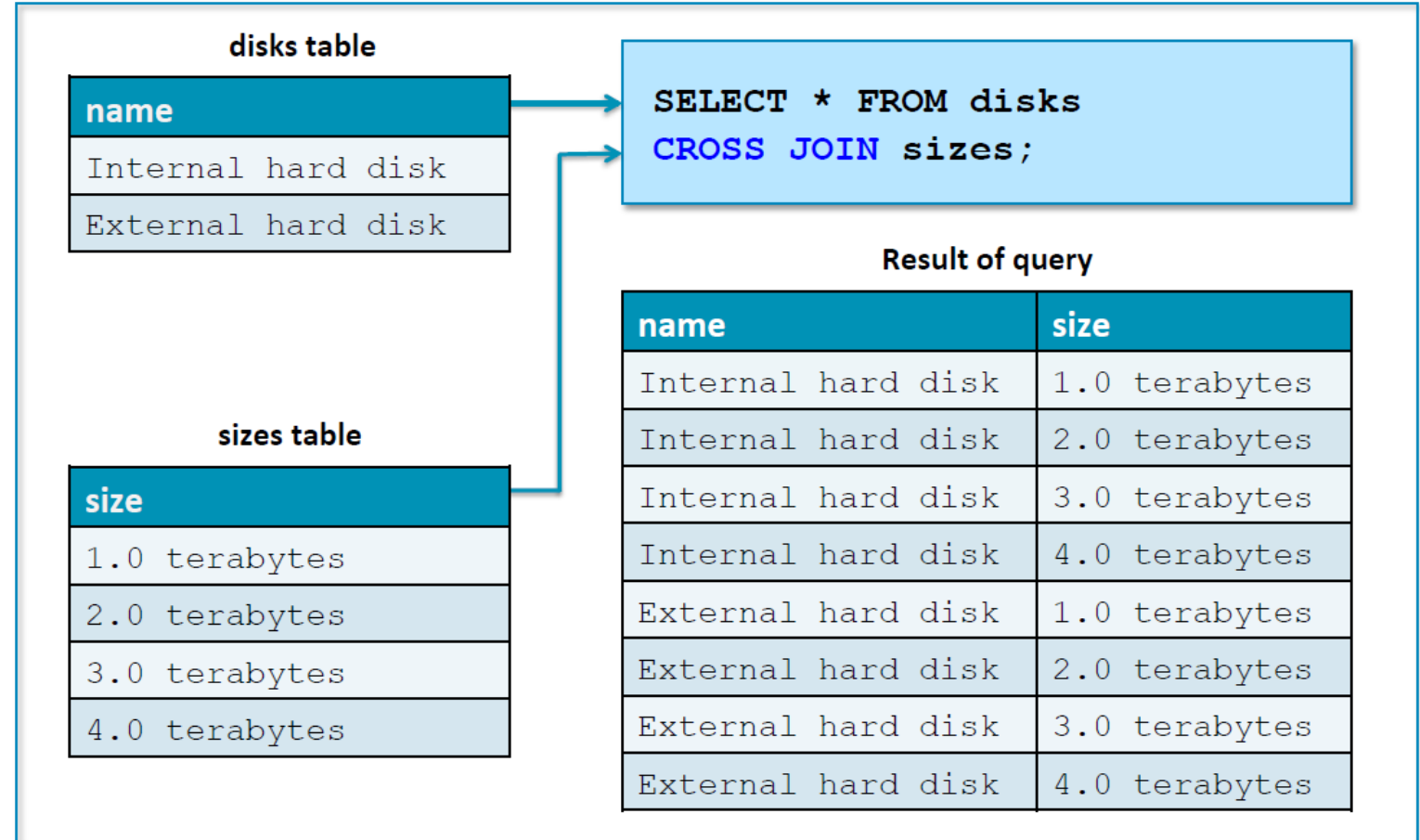
order_id	cust_id	total
1	a	1539
2	c	1871
3	a	6352
4	b	1456
5	z	2137

```
SELECT c.cust_id, name, total
FROM customers c
FULL OUTER JOIN orders o
ON (c.cust_id = o.cust_id)
WHERE c.cust_id IS NULL
OR o.total IS NULL;
```

Result of query

cust_id	name	total
d	Dieter	NULL
NULL	NULL	2137

Cross Join Example



Left Semi Joins

A less common type of join is the *LEFT SEMI JOIN*

- They are a special (and efficient) type of inner join.
- They behave more like a filter than a join.

Left semi joins include additional criteria in the *ON* clause

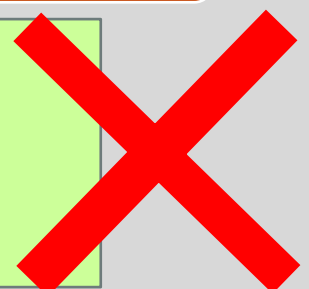
- Only unique records that match these criteria are returned.
- Fields listed in *SELECT* are limited to the left-side table.

```
SELECT c.cust_id FROM customers c LEFT SEMI JOIN orders o ON (c.cust_id =  
o.cust_id AND YEAR(o.order_date) = '2012');
```

Left Semi Joins

Hive does not support IN/EXISTS subqueries.

```
SELECT c.cust_id FROM customers c WHERE o.cust_id  
IN (SELECT o.cust_id FROM orders o WHERE YEAR(o.order_date = '2020'));
```



Using a *LEFT SEMI JOIN* is a common workaround

```
SELECT c.cust_id FROM customers c LEFT SEMI JOIN orders o  
ON (c.cust_id = o.cust_id AND YEAR(o.order_date) = '2020');
```

Hive Common Built-in Functions

Hive offers dozens of built-in functions

- Many are identical to those found in SQL.
- Others are Hive-specific.

Example function invocation

- Function names are not case-sensitive.

```
hive> SELECT CONCAT(fname, ' ') AS fullname FROM customers;
```

To see information about a function

```
hive> DESCRIBE FUNCTION UPPER;  
UPPER(str) – Returns str with all characters changed to uppercase
```

Example Built-in Functions

- These functions operate on numeric values

Function Description	Example Invocation	Input	Output
Rounds to specified # of decimals	ROUND(total_price, 2)	23.492	23.49
Returns nearest integer above	CEIL(total_price)	23.492	24
Returns the nearest integer below	FLOOR(total_price)	23.492	23
Returns absolute value	ABS(temperature)	-49	49
Returns square root	SQRT(area)	64	8
Returns a random number	RAND()		0.584977

Example Built-in Functions

- These functions operate on timestamp values

Function Description	Example Invocation	Input	Output
Convert to UNIX format	UNIX_TIMESTAMP(order_dt)	2020-10-18 18:24:07	1372426023
Convert to string format	FROM_UNIXTIME(mod_time)	1372426023	2020-10-18 18:24:07
Extract date portion	TO_DATE(order_dt)	2020-10-18 18:24:07	2020-10-18
Extract year portion	YEAR(order_dt)	2020-10-18 18:24:07	2020
Returns # of days between dates	DATEDIFF(order_dt,ship_dt)	2020-10-18, 2020-10-20	2

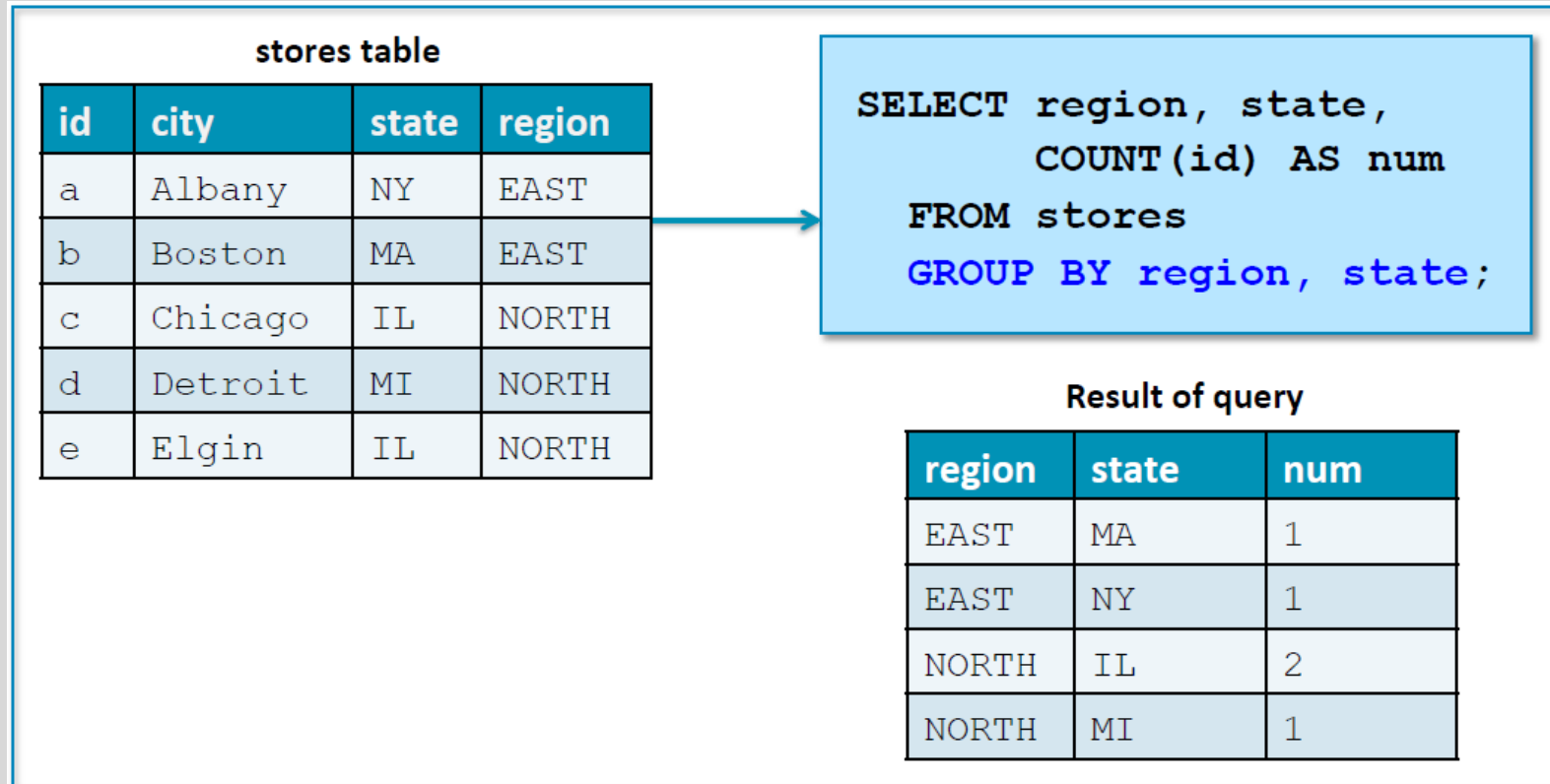
Example Built-in Functions

- Here are some other interesting functions

Function Description	Example Invocation	Input	Output
Convert to uppercase	UPPER(fname)	Bill	BILL
Extract portion of string	SUBSTRING(name, 0, 2)	Alice	Al
Selectively return value	IF(price > 1000, 'A', 'B')	1500	A
Convert to another type	CAST(weight as INT)	3.581	3
Returns size of array or map	SIZE(array_field)	N/A	6

Record Grouping and Aggregate Functions

- **GROUP BY** groups selected data by one or more columns.
 - **Caution:** Columns not part of aggregation must be listed in **GROUP BY**



Built-in Aggregate Functions

- Hive offers many aggregate functions, including

Function Description	Example Invocation
Count all rows	COUNT(*)
Count all rows where field is not null	COUNT(fname)
Count all rows where field is unique and not null	COUNT(DISTINCT fname)
Returns the largest value	MAX(salary)
Returns the smallest value	MIN(salary)
Adds all supplied values and returns result	SUM(price)
Returns the average of all supplied values	AVG(salary)

Brief Summary

Every Hive table belongs to exactly one database

- The *SHOW DATABASES* statement lists databases.
- The *USE* statement switches the active database.
- The *SHOW TABLES* statement lists all tables in a database.

Every column in a Hive table has an associated data type

- Most simple column types are similar to SQL.
- Hive also supports a few complex types.

HiveQL syntax is familiar to those who know SQL.

- A subset of SQL-92, plus Hive-specific extensions.
- Supports inner, outer, and left semi joins.
- Many SQL functions are built into Hive.



HIVE DATA MANAGEMENT

Hive Text File Format

Each Hive table maps to a directory of data, typically in HDFS

- Data stored in one or more files.

Default data format is plain text

- One record per line (record separator is \n)
- Columns are delimited by ^A (field separator is Control-A)
- Complex type elements are separated by ^B (Control-B)
- Map keys/values are separated by ^C (Control-C)

Hive allows you to override these delimiters

- Specify alternate values during table creation

Hive Binary File Format

Hive also supports a few binary formats

- Pro: may offer better performance than text files.
- Con: may limit ability to read data from other programs.

SequenceFile is a Hadoop-specific binary format

- Avoids the need to convert all data to/from strings.
- Supports block- and record-level data compression.

RCFile is a binary format created especially for Hive

- RC stands for **Record Columnar**.
- Column-oriented format efficient for some queries.
- Otherwise, very similar to sequence files.

Creating Databases in Hive

Hive databases are simply namespaces

- Helps to organise your tables.

To create a new database

```
hive> CREATE DATABASE dualcore;
```

To conditionally create a new database

-

```
hive> CREATE DATABASE IF NOT EXISTS dualcore;
```

Creating Table in Hive

Basic syntax for creating a table:

```
CREATE TABLE tablename (colname DATATYPE, ...)
  ROW FORMAT DELIMITED
  FIELDS TERMINATED BY char
  STORED AS {TEXTFILE | SEQUENCEFILE | RCFILE}
```

Creates a subdirectory under `/user/hive/warehouse` in HDFS.

- This is Hive's *warehouse directory*.

Creating Table in Hive

```
CREATE TABLE tablename (colname DATATYPE, ...)
  ROW FORMAT DELIMITED
  FIELDS TERMINATED BY char
  STORED AS {TEXTFILE | SEQUENCEFILE | RCFILE}
```

Specify a name for the table, and list the column names and datatypes (see later)

Creating Table in Hive

```
CREATE TABLE tablename (colname DATATYPE, ...)  
  ROW FORMAT DELIMITED  
  FIELDS TERMINATED BY char  
  STORED AS {TEXTFILE | SEQUENCEFILE | RCFILE}
```

This line states that fields in each file in the table's directory are delimited by some character. Hive's default delimiter is Control-A, but you can specify an alternative delimiter...

Creating Table in Hive

```
CREATE TABLE tablename (colname DATATYPE, ...)
  ROW FORMAT DELIMITED
  FIELDS TERMINATED BY char
  STORED AS {TEXTFILE | SEQUENCEFILE | RCFILE}
```

... for example, tab-delimited data would require that you specify ***FIELDS TERMINATEDBY '\t'***

Creating Table in Hive

```
CREATE TABLE tablename (colname DATATYPE, ...)
  ROW FORMAT DELIMITED
  FIELDS TERMINATED BY char
  STORED AS {TEXTFILE | SEQUENCEFILE | RCFILE}
```

Finally, you can declare the file format. ***STORED AS TEXTFILE*** is the default and does not need to be specified.

Example Table Definition

The following example creates a new table called *jobs*

- Data stored as text with four comma-separated fields per line.

```
CREATE TABLE jobs (id INT, title STRING, salary INT, posted TIMESTAMP)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

- Example of corresponding record for the table above.

```
1,Data Analyst,100000,2020-10-17 14:19:58
```

Complex Column Types

- Recap: Hive has support for complex data types

Column Type	Description
ARRAY	Ordered list of values, all of the same type
MAP	Key-value pairs, each of the same type
STRUCT	Named fields, of possibly mixed types

Creating Tables with Complex Column Types

Complex columns are typed

- Arrays have a single type.
- Maps have a key type and a value type.
- Structs have a type for each attribute.

These types are specified with angle brackets

```
CREATE TABLE stores (store_id SMALLINT, departments ARRAY<STRING>,
    staff MAP<STRING, STRING>,
    address
    STRUCTURE<street:STRING,city:STRING,state:STRING,zipcode:STRING>);
```

Customising Complex Type Storage

We can control which delimiters are used for complex types

```
CREATE TABLE stores (store_id SMALLINT, departments ARRAY<STRING>,
  staff MAP<STRING, STRING>,
  address STRUCTURE<street:STRING,city:STRING,state:STRING,zipcode:STRING>)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '|'
COLLECTION ITEMS TERMINATED BY ',' MAP KEYS TERMINATED BY ':';
```

Row Format Example for Complex Types

The following is an example of a record in that table

1 Audio, Photo A123:Abe, B456:Bob 123 Oak St., Ely, MN, 55731						
id	departments	staff	street	city	state	zipcode

Examples queries on this record

```
hive> SELECT departments[0] FROM stores;
```

Audio

```
hive> SELECT staff['B456'] FROM stores;
```

Bill

```
hive> SELECT address.city FROM stores;
```

Ely

Data Validation in Hive

Hadoop and its ecosystem are 'schema on read'

- Unlike an RDBMS, Hive does not validate data on insert.
 - Files are simply moved into place.
- Loading data into tables is therefore very fast.
- Errors in file format will be discovered when queries are performed.

Missing or invalid data in Hive will be represented as NULL

Loading Data from Files

To load data, simply add files to the table's directory in HDFS

- Can be done directly using *Hadoop fs* commands.
- This example loads data from HDFS into Hive's *sales* table.

```
$ Hadoop fs -mv sales.txt /user/hive/warehouse/sales/
```

Alternatively, use Hive's *LOAD DATA INPATH* statement

- Done from within the Hive shell (or a Hive script)
- This *moves* data within HDFS, just like the command above.
- Source can be either a file or directory.

```
hive> LOAD DATA INPATH 'sales.txt' INTO TABLE sales;
```

Loading Data from Files

Add the *LOCAL* keyword to load data from the local disk.

- Copies a local file (or directory) to the table's directory in HDFS

```
hive> LOAD DATA LOCAL INPATH '/home/training/sales.txt' INTO TABLE sales;
```

- This is equivalent to the following *hadoop fs* command

```
$ hdfs dfs -put /home/training/sales.txt /user/hive/warehouse/sales
```

Loading Data from Files

Add the *OVERWRITE* keyword to delete all records before import

- Removes all files within the table's directory.
- Then moves the new files into that directory.

```
hive> LOAD DATA INPATH '/depts/finance/salesdata' OVERWRITE INTO TABLE sales;
```

Loading Data from Files

Sqoop has built-in support for importing data into Hive

Just add the `--hive-import` option to your Sqoop command

- Creates the table in Hive (metastore)
- Imports data from RDBMS to table's directory in HDFS

```
$ sqoop import
  --connect jdbc:mysql://localhost/dualcore
  --username training
  --password training
  --fields-terminated-by '\t'
  --table employees
  --hive-import
```

Removing Database

Removing a database is similar to creating it.

- Just replace *CREATE* with *DROP*

```
hive> DROP DATABASE dualcore;
```

```
hive> DROP DATABASE IF EXISTS dualcore;
```

These commands will fail if the database contains tables

- Add the *CASCADE* keyword to force removal.
- Caution: this command removes data in HDFS.

```
hive> DROP DATABASE dualcore CASCADE;
```

Removing a Table

Syntax is similar to database removal.

```
hive> DROP TABLE customers;
```

```
hive> DROP TABLE IF EXISTS customers;
```

Caution: These commands can remove data in HDFS!

- **Hive does not have a rollback or undo feature.**

Renaming Tables and Columns

Use *ALTER TABLE* to modify a table's metadata

- This command does not change data in HDFS.

Rename an existing table

```
hive> ALTER TABLE customers RENAME TO clients;
```

Rename a column by specified even through it is unchanged.

```
hive> ALTER TABLE client CHANGE fname first_name STRING;
```


Old Name New Name Type

Modifying and Adding Columns

You can also modify a column's type

- The old and new column names will be the same.
- You must ensure data in HDFS conforms to the new type.

```
hive> ALTER TABLE jobs CHANGE salary salary BIGINT;
```

Old Name New Name Type

Add new columns to a table

- Appended to the end of any existing columns
- Existing data will have *NULL* values for new columns

```
hive> ALTER TABLE jobs ADD COLUMNS (city STRING, bonus INT);
```

Reordering and Removing Columns

Use *AFTER* or *FIRST* to reorder columns

- Ensure that your data in HDFS matches the new order

```
hive> ALTER TABLE jobs CHANGE bonus bonus INT AFTER salary;
```

```
hive> ALTER TABLE jobs CHANGE bonus bonus INT FIRST;
```

Use *REPLACE COLUMNS* to remove columns

- Any column not listed will be dropped from metadata

```
hive> ALTER TABLE jobs REPLACE COLUMNS (id INT, title STRING, salary INT);
```

Controlling Table Data Location

Hive stores data associated with a table in its warehouse directory

Storing data below Hive's warehouse is not always ideal

- Data might be shared by several users

Use *LOCATION* to specify directory where table data resides

```
CREATE TABLE adclicks (campaign_id STRING, when TIMESTAMP, keyword  
STRING, site STRING, placement STRING, was_clicked BOOLEAN, cost  
SMALLINT) LOCATION '/dualcore/ad_data';
```

Self-Managed (External) Tables

Recall that dropping a table removes its data in HDFS

Using *EXTERNAL* when creating the table avoids this behaviour

- Dropping an external table removes only its metadata.

```
CREATE EXTERNAL TABLE adclicks (campaign_id STRING, when TIMESTAMP,  
keyword STRING, site STRING, placement STRING, was_clicked BOOLEAN,  
cost SMALLINT) LOCATION '/dualcore/ad_data';
```

Simplifying Complex Queries

Complex queries can become cumbersome

- Imagine typing this several times for different orders

```
SELECT o.order_id, order_date, p.prod_id, brand, name FROM orders o  
  JOIN order_details d ON (o.order_id = d.order_id)  
  JOIN products p ON (d.prod_id = p.prod_id) WHERE o.order_id=123456;
```

Creating Views

Views in Hive are conceptually like a table, but backed by a query

- You cannot directly add data to a view

```
CREATE VIEW order_info AS  
SELECT o.order_id, order_date, p.prod_id, brand, name FROM orders o  
  JOIN order_details d ON (o.order_id = d.order_id)  
  JOIN products p ON (d.prod_id = p.prod_id);
```

Our query is now greatly simplified

```
hive> SELECT * FROM order_info WHERE order_id=123456;
```

Inspecting and Removing Views

Use *DESCRIBE FORMATTED* to see underlying query

```
hive> DESCRIBE FORMATTED order_info;
```

Use *DROP VIEW* to remove a view

```
hive> DROP VIEW order_info;
```

Saving Query Output to a Table

SELECT statements display their results on screen

To send results to a Hive table, use *INSERT OVERWRITE TABLE*

- Destination table must already exist
- Existing contents will be deleted

```
hive> INSERT OVERWRITE TABLE ny_customers SELECT * FROM customers  
WHERE state = 'NY';
```

INSERT INTO TABLE adds records without first deleting existing data

```
hive> INSERT INTO TABLE ny_customers SELECT * FROM customers  
WHERE state = 'NJ' OR state = 'CT';
```

Creating Tables based on Existing Data

Hive supports creating a table based on a *SELECT* statement

- Often known as 'Create Table As Select' (CTAS)

```
CREATE TABLE ny_customers AS SELECT cust_id, fname, lname FROM  
customers WHERE state='NY';
```

Column definitions are derived from the existing table

Column names are inherited from the existing names

- Use aliases in the *SELECT* statement to specify new names

Writing Output to a Filesystem

You can save output to a file in HDFS

```
hive> INSERT OVERWRITE DIRECTORY '/dualcore/ny/' SELECT * FROM  
customers WHERE state = 'NY';
```

Add *LOCAL* to store results to local disk instead

```
hive> INSERT OVERWRITE LOCAL DIRECTORY '/dualcore/ny/' SELECT * FROM  
customers WHERE state = 'NY';
```

Both product text files delimited by Ctrl-A characters

Writing Output to HDFS, Specifying Format

To write the files to HDFS with a user-specified format:

- Create an external table in the required format.
- Use *INSERT OVERWRITE TABLE*.

```
hive> CREATE EXTERNAL TABLE ny_customers  
      (cust_id INT, fname STRING, lname STRING) ROW FORMAT DELIMITED  
      FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION  
      '/dualcore/nydata';
```

```
hive> INSERT OVERWRITE TABLE ny_customers SELECT cust_id, fname, lname  
      FROM customers WHERE state = 'NY';
```

Multi-Table Insert

We just saw that you can save output to an HDFS file.

```
INSERT OVERWRITE DIRECTORY 'ny_customers'  
SELECT cust_id, fname, lname FROM customers WHERE state = 'NY';
```

This query could also be written as follows

```
FROM customers c  
  INSERT OVERWRITE DIRECTORY 'ny_customers'  
  SELECT cust_id, fname, lname WHERE state = 'NY';
```

Multi-Table Insert

We sometimes need to extract data to multiple tables

- Hive *SELECT* queries can take a long time to complete.

Hive allows us to do this with a single query

- Much more efficient than using multiple queries

The following example demonstrates multi-table insert

- Result is two directories in HDFS

```
FROM customers c
  INSERT OVERWRITE DIRECTORY 'ny_customers'
  SELECT fname, lname WHERE state = 'NY';

  INSERT OVERWRITE DIRECTORY 'ny_count'
  SELECT count(DISTINCT cust_id) WHERE state = 'NY';
```

Multi-Table Insert

The following query produces the same result

```
FROM (SELECT * FROM customers WHERE state='NY') nycust  
  INSERT OVERWRITE DIRECTORY 'ny_names' SELECT fname, lname  
  INSERT OVERWRITE DIRECTORY 'ny_count'  
    SELECT count(DISTINCT cust_id);
```

Controlling Access to Data – Hive Authentication

HiveServer2/3 can be configured for Kerberos or LDAP authentication

- You must provide a user ID and password when connecting to HiveServer2/3 from Beeline

```
[training@localhost analyst]$ beeline
Beeline version 0.10.0-cdh4.2.1 by Apache Hive

beeline> !connect jdbc:hive2://localhost:10000
training mypwd org.apache.hive.jdbc.HiveDriver

Connecting to jdbc:hive2://localhost:10000
Connected to: Hive (version 0.10.0)
Driver: Hive (version 0.10.0-cdh4.2.1)
Transaction isolation: TRANSACTION_REPEATABLE_READ

beeline>
```

Hive Authorisation with Apache Sentry

Sentry is a plug-in for enforcing authorisation

Sentry can be enabled in HiveServer2/3

- And in Impala

Sentry provides fine-grained, role-based authorisation for Hive data and metadata

Sentry was developed at Cloudera

- Available starting with CDH 4.3
- Project is in incubation status at Apache
- <http://incubator.apache.org/projects/sentry.html>

Sentry Access Control Model

What does Sentry control access to?

- Server
- Database
- Table
- View

Who can access Sentry-controlled objects?

- Users in a Sentry role
 - Sentry roles = one or more groups

How can role members access Sentry-controlled objects?

- Read operations (*SELECT* privilege)
- Write operation (*INSERT* privilege)
- Metadata definition operations (*ALL* privilege)

Essential Points

Each Hive table maps to a directory in HDFS

- Table data stored as one or more files.
- Default format: plain text with delimited fields.
- Binary formats may offer better performance but limit compatibility.

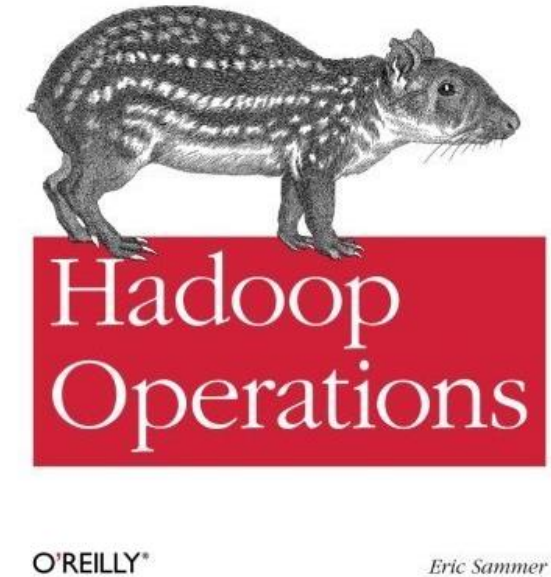
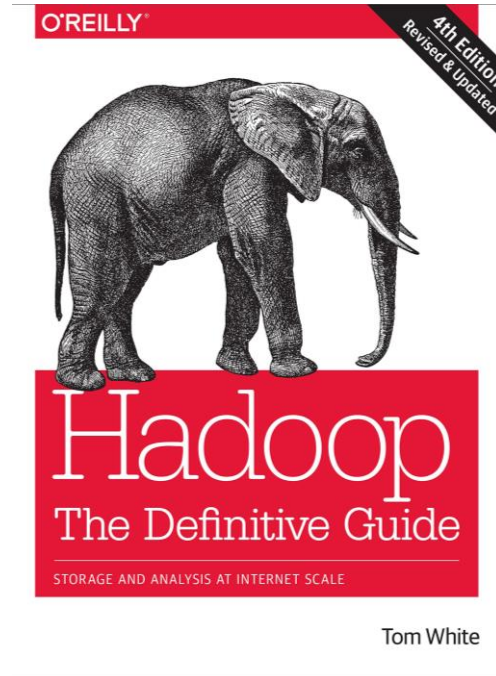
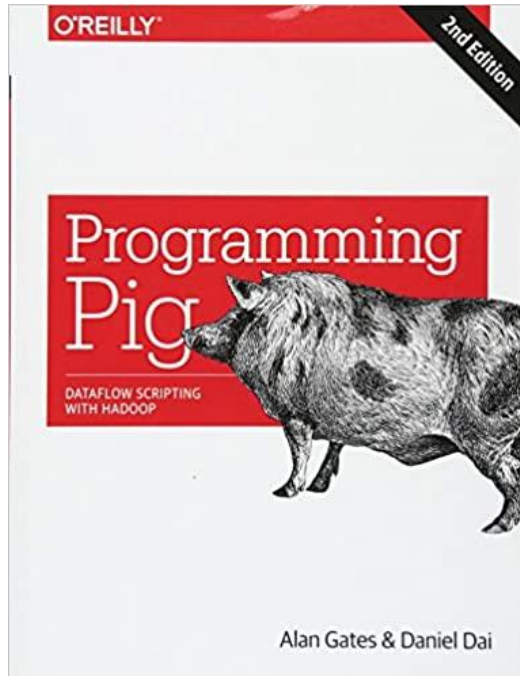
Dropping a Hive-managed tables deletes data in HDFS

- External tables require manual data deletion.

ALTER TABLES is used to add, modify, and remove columns.

Views can help to simplify complex and repetitive queries.

In a secure environment, Sentry provides Hive authorisation.



Texts and Resources

- Unless stated otherwise, the materials presented in this lecture are taken from:
 - Hadoop: the definitive guide, White, Tom (Tom E.) author., 4th ed., Sebastopol, California: O'Reilly, 2015.
 - Hadoop operations, Sammer, Eric., Loukides, Michael Kosta.; Nash, Courtney.; Romano, Robert (Illustrator), illustrator., First edition., Sebastopol, CA: O'Reilly, 2012.
 - Programming pig: dataflow scripting with hadoop, Gates, Alan, author.; Dai, Daniel, 2nd edition., Beijing, China: O'Reilly, 2017



Unit Summary

- Each Hive table maps to a directory in HDFS
 - Table data stored as one or more files
 - Default format: plain text with delimited fields
 - Binary formats may offer better performance but limit compatibility
- Dropping a Hive-managed tables deletes data in HDFS
 - External tables require manual data deletion
- ALTER TABLE is used to add, modify, and remove columns
- Views can help to simplify complex and repetitive queries
- In a secure environment, Sentry provides Hive authorisation

